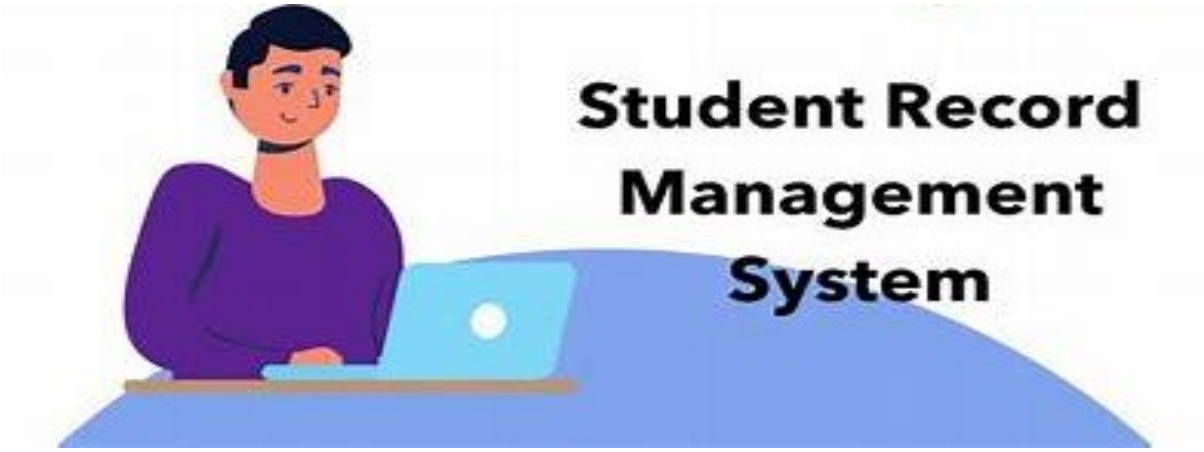


Student Management System

Submitted By: ATHARV PRIYANKAR

Registration Number: 26BSA10026

Branch: B.Tech CSE [CLOUD COMPUTING AMD AUTOMATION]

Course: Introduction to Problem Solving in Python

Institution: VIT Bhopal University

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced

2. Introduction

The Student Management System is a simple Python project made to manage student details and marks. In this project, we can add student information, enter marks for three subjects, calculate total marks and percentage, and find the student's grade.

I developed this project to understand how Python can be used to solve simple real-life problems. While making this project, I used basic Python concepts like loops, functions, lists, modules, and classes.

3. Problem Statement

In schools and colleges, student details and marks are often managed manually. This can take a lot of time and may sometimes lead to mistakes.

To solve this problem, I created a Student Management System that stores student details, calculates marks and percentages, assigns grades, and helps users find student records easily.

4.FUNCTIONAL REQUIREMENT

Functional requirements describe what the system should do.

1. Add Student: Enter the student's name, registration number, and subject-wise marks.
2. Display Records: Show all stored student details.
3. Search Student: Find a student using their registration number.
4. Calculate Total: Calculate the total marks of three subjects.
5. Calculate Percentage: Calculate the percentage based on the marks entered.
6. Assign Grade: Assign a grade according to the percentage.
7. Exit: Allow the user to close the application.

5. Non-functional Requirements

Non-functional requirements describe how the system should perform.

-Usability: The system should have a simple, menu-driven interface.

-Performance: The system should process student records quickly.

-Reliability: Calculations should produce correct results for valid inputs.

-Maintainability: The code should be divided into separate modules for easy updates.

-Portability: The application should run on systems with Python installed.

-Simplicity: The system should be easy for beginners to understand and use.

6. System Architecture

The project is divided into three Python files. Each file has a different role.

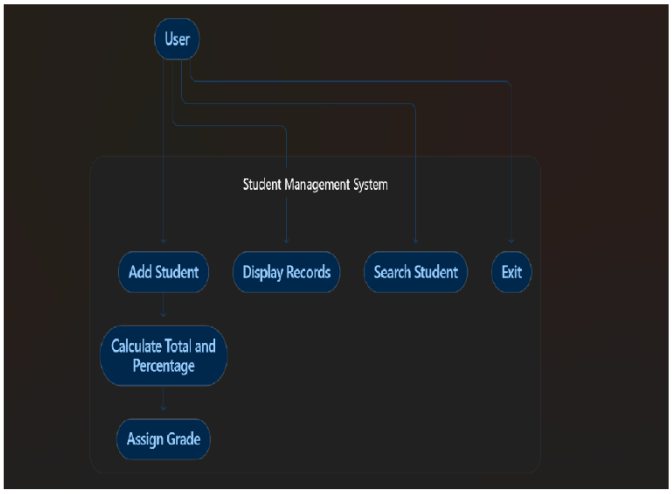
- **main.py:** This is the main file. It displays the menu and takes input from the user.
- **student.py**: This file contains the Student class and calculates total marks, percentage, and grades.
- **records.py:** This file stores student records and helps in adding, displaying, and searching for students.

All three files work together to run the Student Management System.

7.Design Diagrams

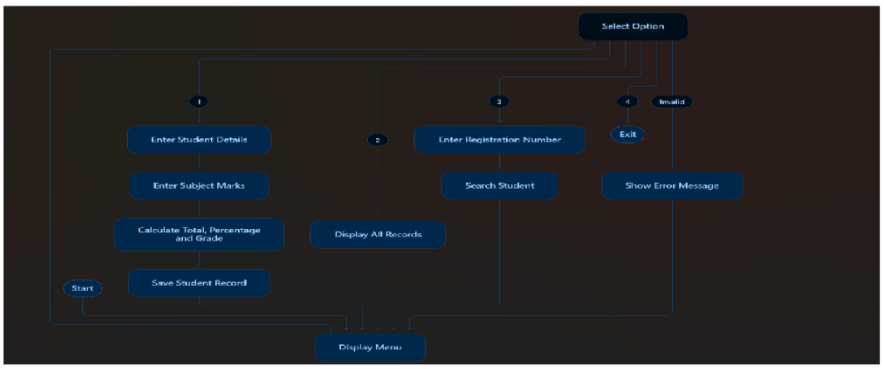
7.1 Use Case Diagram

This diagram shows the different activities a user can perform in the system, such as adding a student, displaying records, and searching for a student.



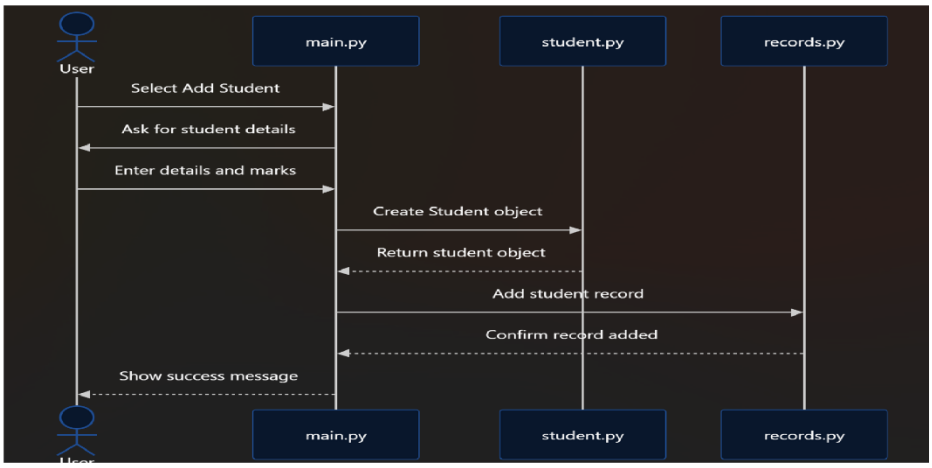
7.2 Workflow Diagram

This diagram shows how the program works from the beginning until the user exits.



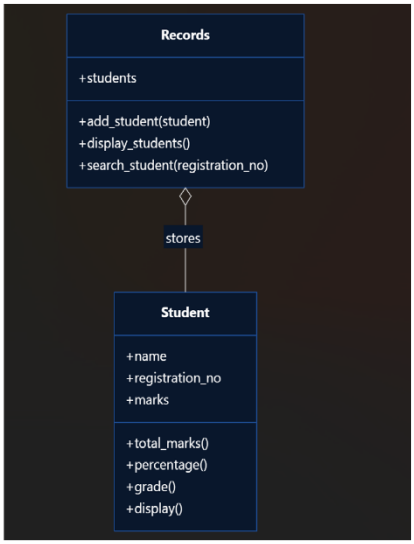
7.3 Sequence Diagram

This diagram shows how the different files interact when a new student is added.



7.4 Class Diagram

This diagram shows the Student class and the functions used to manage student records.



7.5 ER Diagram

An ER diagram is not included in this project because I have not used a database. The student records are stored in a Python list and are available only while the program is running.

If I add a database in the future, I can create an ER diagram to show the relationship between students and subjects.

8. Design Decisions & Rationale

While making this project, I chose simple Python concepts so that the code would be easy to understand.

Design choice	Reason
Python	Easy to learn and use
Separate files	Keeps the code organized
Student class	Stores the details of each student
List	Stores multiple student records
Registration number	Helps find a particular student
Menu-driven system	Makes the program easy to operate
Temporary storage	Keeps the project simple without a database

9.IMLEMENTATION

The project is divided into three files.

9.1 main.py

This is the main file of the project. It displays the menu, takes input from the user, and calls the required functions from other files.

9.2 student.py

This file contains the Student class. It stores the student's name, registration number, and marks. It also calculates total marks, percentage, and grade.

9.3 records.py

This file manages the student records. It allows the user to add a new student, display all records, and search for a student using the registration number.

9.4 Subjects Used

The project includes marks for these three subjects:

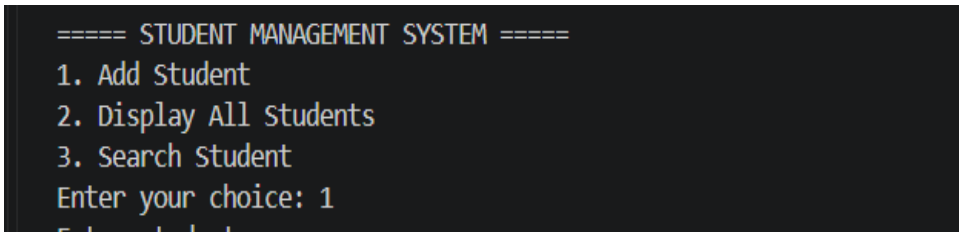
1. Environmental Sustainability
2. Calculus
3. Introduction to Python

The system uses these marks to calculate the total, percentage, and grade of each student.

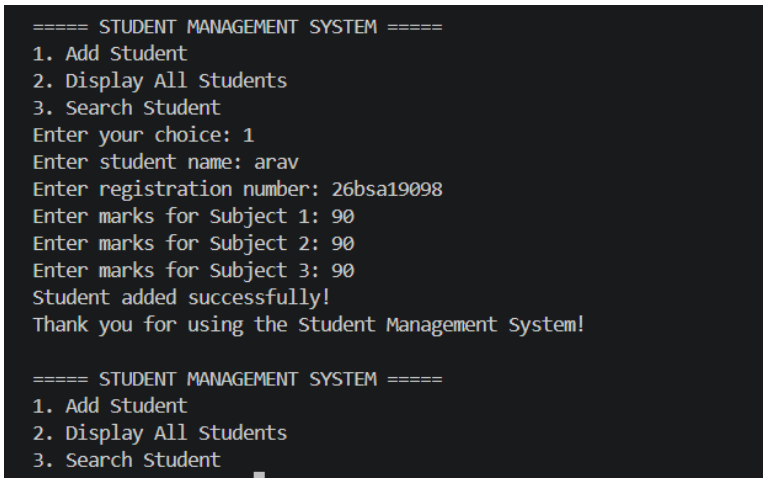
9. Screenshots / Results

The following screenshots can be added to show how the project works.

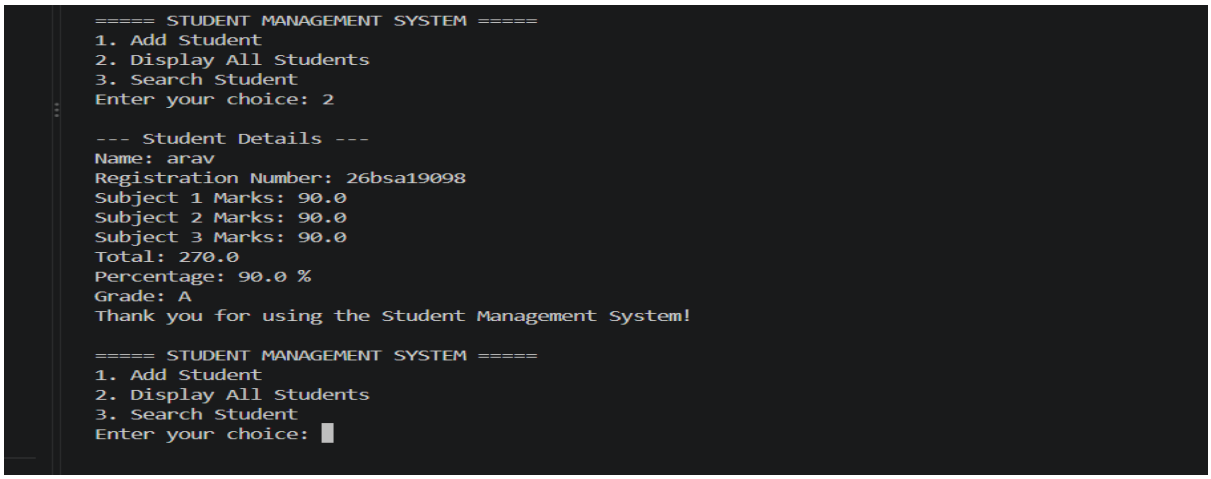
Screenshot 1: MAIN Menu



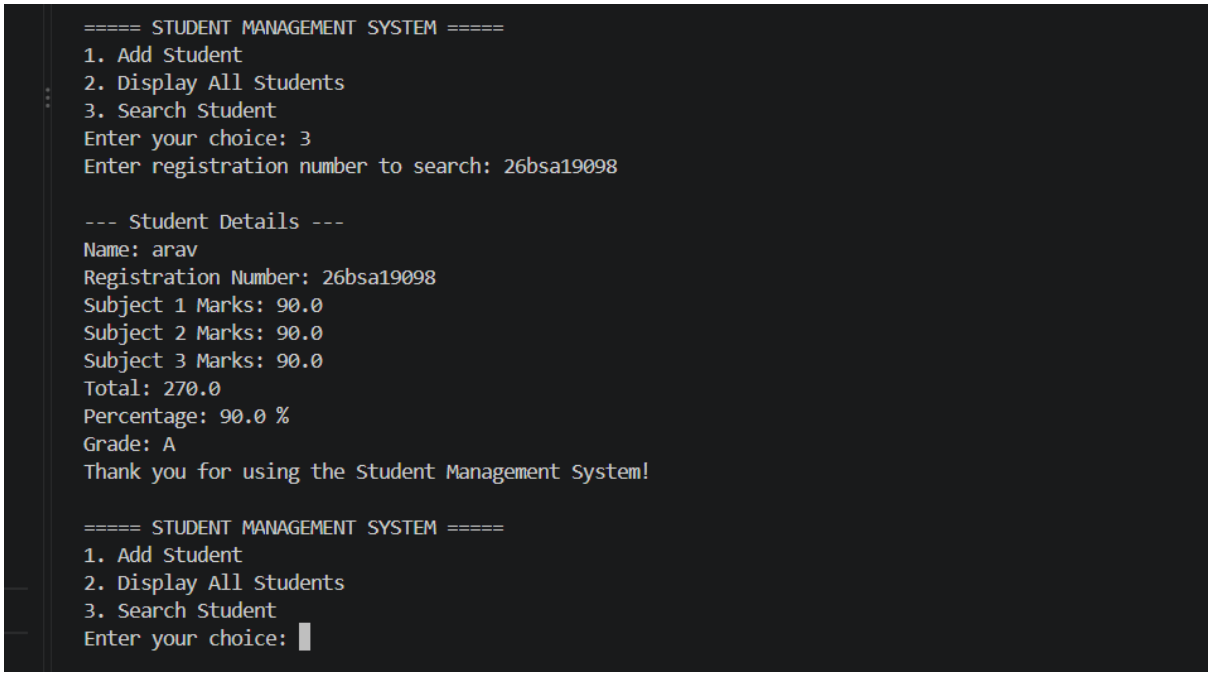
Screenshot 2: Adding a Student



Screenshot 3: Displaying Student Records



Screenshot 4: Searching for a Student



11. Testing Approach

I tested the project using different inputs to check whether all the features were working properly.

Test case	Expected result
Add a student	Student should be added successfully
Display records	All saved students should be displayed
Search a valid registration number	Correct student details should appear
Search an invalid registration number	"Student not found" message should appear
Enter subject marks	Total, percentage, and grade should be calculated
Exit the program	Program should close successfully

Testing was done manually by running the program in the Python terminal.

12. Challenges Faced

While developing this project, I faced a few challenges:

1. Import Errors: I had some problems while importing classes and modules from different files.
2. Case Sensitivity: I learned that Python treats uppercase and lowercase letters differently.
3. Managing Records: I learned how to store student objects in a list and search for them using registration numbers.
4. Subject Names: I changed the subject numbers to actual subject names to make the output easier to understand.
5. Connecting Files: I learned how to connect multiple Python files and make them work together.

13. Learnings & Key Takeaways

By working on this project, I learned how to use Python for a simple real-life application.

Some of the main things I learned are:

- Using variables, operators, and data types.
- Working with loops and conditional statements.
- Creating functions and using lists.
- Understanding classes and objects.
- Dividing code into different modules.
- Storing and searching student records.
- Calculating marks, percentages, and grades.
- Finding and correcting errors in Python code.

14. Future Enhancements

There are a few features that I would like to add to this project in the future:

1. Database: Store student records permanently so they are not lost after closing the program.
2. Update and Delete: Allow users to edit or delete student records.
3. Graphical Interface: Make the project easier to use by adding a graphical interface.
4. More Subjects: Allow users to add more subjects.
5. Input Validation: Prevent incorrect marks and duplicate registration numbers.
6. Report Card: Generate a report card for each student.
7. Export Records: Save student records in CSV or Excel format.